Lab Assignments
Computer Graphics DT519A,
Örebro University, autumn 2024

October 28, 2024

1 Projective rendering in OpenGL

In this lab you will first learn how to communicate with the graphics hardware, and then learn the foundations of how to put 3D objects where you want them on the screen; first with some manual labour (to see some of what goes on under the surface), and then with tools that simplify life to some extent.

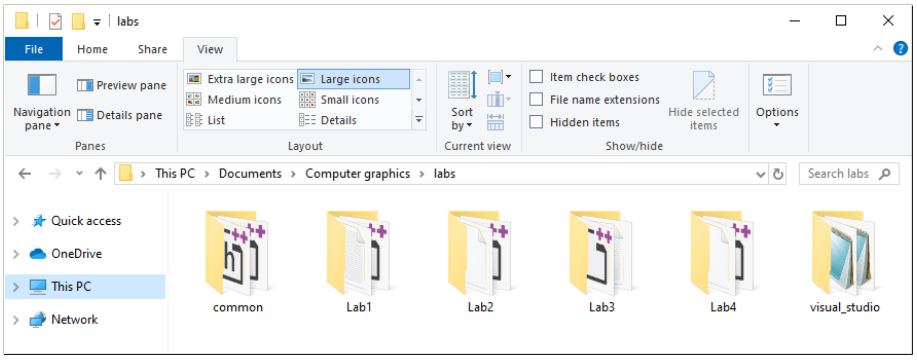
While working with this lab, it will be useful to have the OpenGL reference pages at hand: https://registry.khronos.org/OpenGL-Refpages/gl4/. The OpenGL Wiki at https://www.opengl.org/wiki can also be useful.

Learning goals

- Introduction to OpenGL:
  - communicate with the graphics card,
  - drawing triangles,
  - understanding depth and frame buffers,
  - using basic vertex and fragment shaders.
- 3D geometry and algebra:
  - model and view transformations for rotating and placing objects,
  - perspective transformations.

Getting started with the code skeleton

First, download the code from blackboard and unzip it in a location of your choice. Important: both the `common` and `Lab1` folders should share *the same parent folder* (as well as `visual_studio` if you want to use that as opposed to VS Code or other IDEs). (See the file structure in the figure below.)



Windows If you use Windows and Visual Studio (not VS Code), find the `visual_studio` directory and open the `lab1.sln` solution file in Visual Studio. The Visual Studio solution files are made for VS 2015, which is installed in the lab room (T006). If you have a more recent version of Visual Studio, you will probably be asked to update the solution file (which you should do). The selected target configuration should be "Debug" and "x64". Right-click the "lab1-1" project and select "build" to compile it. It should compile without errors. The executable files will be located in a build directory under the "Lab1" folder. To see the printed output from your program in this task, it may be necessary run the program as "*Start without Debugging*" (or to use the windows command prompt, `cmd.exe`).

If you're not a fan of Visual Studio, you are also welcome to try **CLion** which is a very capable IDE, or **VS Code**.

Linux If you use Linux, you will first have to install the two dependencies `libglew-dev` and `libglfw3-dev`. You should also install `libglm-dev` for the later tasks. Then, run `cmake` followed by `make` in a subdirectory relative to where you downloaded the source files, like so:

```
~/code/computer_graphics/labs/Lab1$ mkdir build
~/code/computer_graphics/labs/Lab1$ cd build
~/code/computer_graphics/labs/Lab1/build$ cmake ..
~/code/computer_graphics/labs/Lab1/build$ make
```

The executable should appear in the build directory and can be executed from there.

```
~/code/computer_graphics/labs/Lab1/build$ ./lab1-1
```

Mandatory

1.1 Your first triangle

Check the provided code skeleton for the first task (`lab1-1.cpp`). This file provides some boiler-plate code for getting the basic things up and running.

In the `main` function, we first initialise the GLFW library, ask the window manager to give us a window to draw in, set callback functions so that our program can react to window resizing and closing, and use GLEW to get access to the OpenGL functions we will use to actually draw something.

In the skeleton code for this task you will also find two strings called a "vertex shader" and a "fragment shader". These are the programs that will run on the GPU and are compiled at run-time by the graphics driver. Without them, it will not be possible to do anything with the GPU. However, you won't actually touch the shader code until Lab 1.2.

Tasks

Your task is to draw a single triangle in a window, as shown in Figure 1. Even with the given boiler-plate code, there is still a number of steps that need to be performed in your CPU code (`lab1-1.cpp`) in order to tell the GPU how to draw the triangle.

1. Begin by defining the vertices (that is, corners) of your triangle (for example, at the "create geometry" placeholder of the skeleton code). This you can do simply by creating an array of floats `float points[]`, with enough elements to store the coordinates of the three vertex positions of a triangle. This is a normal C-style array of floats that lives on the CPU

side. It will be passed to the vertex shader program, where each vertex position will be copied to the shader variable `vp` (defined in the vertex shader code), and executed on the GPU. (How many elements do you need in the `points` array? Hint: How many points are there, and how many floats do you need per point?) Keep the geometry values in the range $[-1, 1]$ to remain within the limits of the screen.

2. Create a vertex array object (VAO). (Recall from the lecture that data is passed to the GPU using a VAO, which in turn holds pointers to *buffer objects*.) In practice, your VAOs and VBOs will be represented by a simple *handle* that is just an unsigned integer (`GLuint`), which is managed by the functions listed below.

   The OpenGL wiki on vertex specification[1] describes how you can create and enable a VAO.

   Start with defining a handle `GLuint vao;`. The safest (backwards compatible) way of setting up a VAO is to first *generate* one, using a call to `glGenVertexArrays(1, &vao)`, and then *bind* it to make it the currently active one: `glBindVertexArray(vao)`. (If your GPU uses OpenGL 4.5 or later, you can also use *direct state access* functions which don't require binding objects before use, but for compatibility reasons, these instructions will not cover direct state access.)

   After creating/binding the VAO, you can issue other calls to modify it, thus determining what will happen when issuing a draw call using this VAO.

3. Create a vertex buffer object (VBO) that will hold the data for the vertices (that is, the points you created above).

   As with the VAO, you first define a `GLuint vbo;` handle and pass a pointer to it to the `glGenBuffers` function. Then you bind it (make it current) with `glBindBuffer`. The function `glBindBuffer` takes as input a certain kind of *target*. Keep in mind that at this point we are creating a buffer object to hold *vertex attributes* (specifically, the *position* attributes of the corners of our triangle).

4. After creating the VBO, you must also copy some data to it (from the CPU to the GPU side). In this case, the data in the VBO is the attribute data that defines the vertex positions.

   This is done with the `glBufferData` function, which takes four arguments. The first (*target*) is the same kind of target as for `glBindBuffer` (still about vertex attributes). The second (*size*) is the number of bytes to be copied (that is, the size of your array of point coordinates). The third (*data*) is a pointer to the array to be copied. The fourth (*usage*) is less important, but can impact performance in larger programs. In this case, `GL_STATIC_DRAW` is a sensible choice.

5. *Enable* each vertex attribute your program needs. So far, the only attribute that our vertices have is position, but later on they might have normal vectors, colours, texture coordinates, and other attributes too.

   By default, all attributes are disabled, so you have to make a call to `glEnableVertexAttribArray` for each one you want to use.

[1] https://www.opengl.org/wiki/Vertex_Specification

So which index should you specify in the enable function? This is specified in the vertex shader code: `layout(location=0)in vec3 vp;` (By the way, unless something else is specified, the first attribute always has index, or location, 0.)

This boils down to simply calling `glEnableVertexAttribArray (0);`

6. At this point, we have a VAO where attribute 0 has been enabled. We also have a VBO that holds our vertex data.

   In our case, the VBO only holds a continuous sequence of position data. However, the buffer storage is flexible and can be set up in any number of ways. Therefore the GPU needs to know how many bytes are needed for each attribute component (float/double etc), how many components the attribute has (is it a 2D position, a 3D colour vector, or something else?), and how the attributes are ordered in the buffer (are they contiguous or interleaved?). (Recall how this was done in the lecture.)

   This setup is stored in the VAO, and is performed using one more function call: `glVertexAttribPointer`. Refer to the OpenGL docs and the lecture slidees.

   All this work probably sounds overly complicated, just for drawing a triangle! But for a more complex scene, with multiple objects, multiple materials and shader programs, it will be very convenient to have the flexibility to arrange your vertex attribute data in one or more VBOs as you like, and to configure sets of attributes in VAOs so that you can then easily switch from VAO to the next when drawing your scene to the screen.

7. Finally tell the GPU to draw the triangle data you have provided. This is done further down in the code: in the main event loop (see placeholder in the skeleton code), using a call to `glDrawArrays`. (You'll want to draw a set of triangles, starting at index 0, using 3 vertices.)

   Before your draw call, you also need to *bind* your VAO by calling the function `glBindVertexArray( vao )`. Once you have done that, `glDrawArrays` will fetch data for the vertices it will draw from the buffer objects that are specified in your VAO. In larger programs, one may bind a new VAO for each object between each draw call, but we only have one VAO so we can simply leave it bound.

When you have solved the task you should see something similar to Figure 1, though your vertices may be placed differently.

Mandatory

1.2 Introduction to shaders

In this task you will pass information between shaders and write a little GLSL code. Recall from the lecture that vertex shaders, as the name implies, are executed on each vertex. Fragment shaders are executed on elements that (might) contribute to the final color of each pixel, in parallel.

Start by copying your solution of the previous task (that is, the code for specifying the triangle, your VBO and VAO, and the drawing command) into the indicated places in `lab1-2.cpp`.

Note that the shaders are now loaded from separate text files (at the top of the cpp file) instead of being hard-coded. This has the advantage of not requiring you to recompile your CPU program to make changes to the shader code run on the GPU.

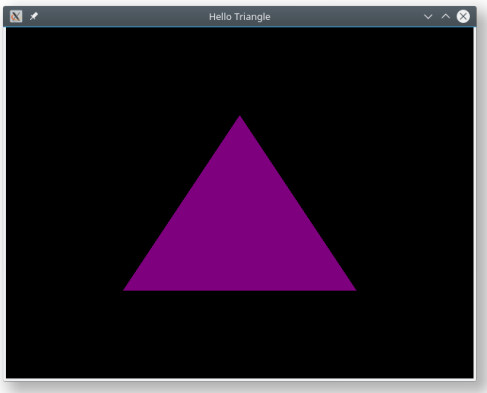


Figure 1: A purple triangle drawn in Lab 1.1.

The skeleton code has a new function, called `checkShaderCompileError`, which is very convenient. This function is called after each call to `glCompileShader` to show debugging information on `stdout` in case you make a mistake in the shader source code.

There `key_callback` function has also been extended to reload and recompile the shader programs when the "R" key is pressed. This way, you don't have to recompile and restart the C++ program every time you want to edit the shader code. You can just hit "R".

Tasks

1. The CPU code (from the previous task) sends the data for the position attribute of the vertices to the `vp` variable in the vertex shader. Now, have a look at the source code for the vertex shader (now in `lab1-2_vs.glsl`) and the fragment shader (now in `lab1-2_fs.glsl`).

   Note that the vertex shader writes output to the special `gl_Position` output variable. Recall that this variable will be used by the rasterisation stage to determine at which pixel the vertex ends up. (Also note that `gl_Position` is defined as a 4D vector (`vec4`). Simply keeping the fourth element as 1.0 is sufficient for now.)

   You may also pass data in your own variables from the vertex to the fragment shader (as well as other shader stages) by declaring an `out` variable from the vertex shader and a corresponding `in` variable in the fragment shader.

   Modify the vertex shader (`lab1-2_vs.glsl`) as follows. Declare a global variable `out vec3 position` In the `main` function, write the position that you get in `vp` to it – so that the fragment shader (that comes after the vertex shader) also can read it.

2. Modify the *fragment shader* (`lab1-2_fs.glsl`) to declare an `in vec3 position` variable and use its value (which it gets from the vertex shader) when setting the `frag_colour` output variable. so that the colour varies with the position of the pixels, instead of being fixed to the colour (0.5, 0.0, 0.5).

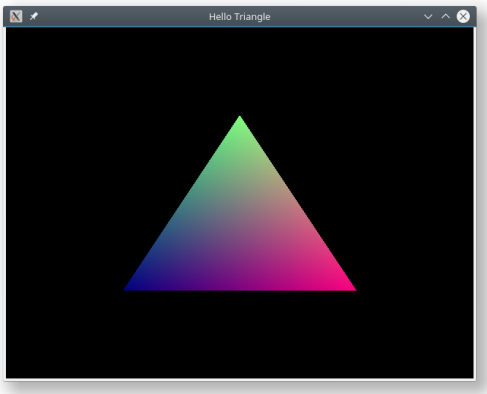


Figure 2: A multicolored triangle drawn in Lab 1.2.

You may notice that some parts of your triangle are black, after you have done this. Fix this by modifying the output from the vertex shader (e.g., adding an offset). You should see something similar to Figure 2 when done.

Make sure you understand why you get the colours that you do. For example: why is one corner greener than the others, and why is one corner black?

Mandatory

1.3 Passing parameters to shader programs

You might be wondering how computer games and other interactive applications work, if the visible content is to respond to the user's inputs.

Suppose we want the triangle to move when we press the arrow keys on the keyboard. Do we change the coordinates of the triangle and upload new values to the GPU (with `glNamedBufferData`) at each iteration of the event loop? This would work for a single triangle, but would be very expensive when we have triangles in the millions.

In practice, we will keep the geometry definition as it is, and send the desired *transformation* as a parameter to the GPU instead. This is done using `uniform` variables;[2] that is, global variables that are the same for all executions of the shader program. (Lecture 2 also mentions `uniform` variables.)

Tasks

1. Create one or more (global) variables in your C++ program to hold x and y offsets, which will be used to move the triangle. (You can also have other modifiers, to tweak the scale and colour, etc.)

2. Update your `key_callback` to read keys and update these variables. Have the left and right keys change the horizontal offset while up and down keys change the vertical offset.

3. Now, to use these offsets on the GPU, we need to create a `uniform` variable in the shader program, and put the current values there. (Recall that it is

[2] https://www.opengl.org/wiki/Uniform_(GLSL)

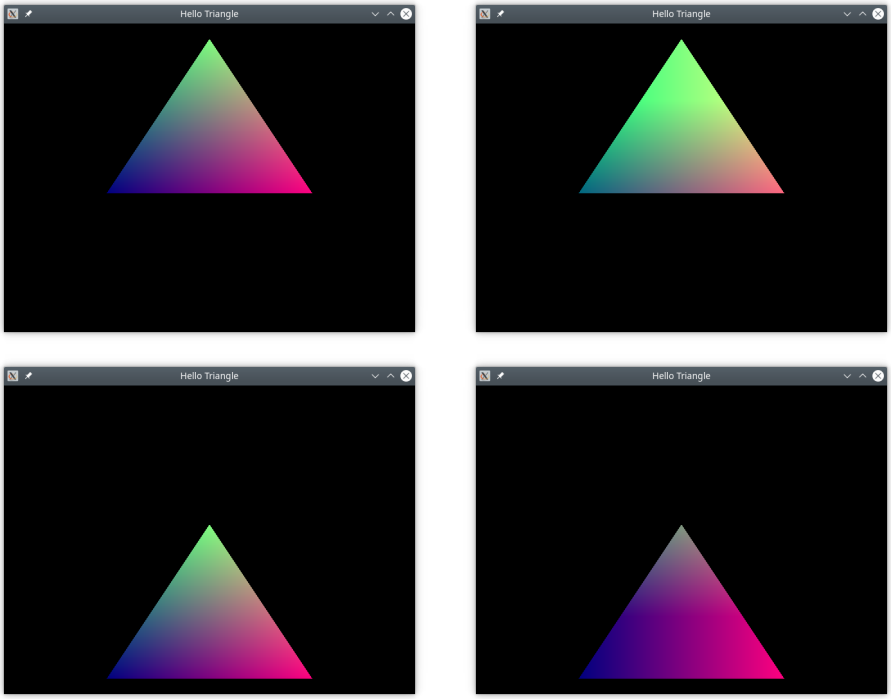


Figure 3: Lab 1.3, moving the triangle. *Left:* colours fixed at triangle. *Right:* colours constant in window.

called "uniform" because it applies the same to all vertices, so is not a per-vertex attribute like the data that you put in the VBO.)

Declare a `uniform vec2 position_offset;` in the *vertex shader*.

4. On the CPU side, you will need to query the GPU for the location of the uniform and then write a value to it. Use `glGetUniformLocation` to get the location, and one of the `glUniform*` functions to write to it. Have a look at the documentation for these functions in the OpenGL API reference. (You can also get the current value with `glGetUniform`.)

   The first parameter of `glGetUniformLocation` is the shader program (the variable `shader_program` in the skeleton code) and the second is a string with the name of the uniform in the shader code.

   There are many variants of the `glUniform` functions, since OpenGL is written in C and does not support overloading. Since your `position_offset` is a `vec2`, you should use `glUniform2f` (that is, sending two floats).

5. On the shader side, use the `position_offset` to modify `gl_Position` and your own `position`. It will be convenient to use *swizzle* operators to update the $x/y/z$ values independently: e.g., `vec3(vp.x + offset.x, vp.y, vp.z)`

6. Make sure that you can implement *two versions* of this program: one where the colour of the triangle is "attached to the triangle" as the triangle moves, and one where the colour is "attached to the window" so that the triangle's colours change when it moves. (Hint: `gl_Position` determines where the triangle is rasterised on screen, and the variable you pass to the fragment shader can determine how it is coloured.) See Figure 3.

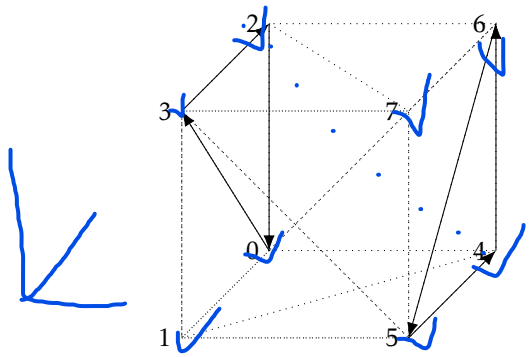


Figure 4: Indexing the faces of a cube. Note the counter-clockwise ordering of the faces 0–2–3 and 4–5–6 on opposite sides of the cube.

7. Optionally, declare a `uniform float modifier` parameter in your *fragment shader* and make the scroll wheel (or some other key) change colours of the triangle depending on the value of the modifier.

Mandatory

1.4 3D geometry

So far, we have mostly been disregarding the three-dimensional nature of the data and concerned ourselves only with the two dimensions of the screen. This changes now.

Start by defining an array of vertex positions that make up the corners of a cube. (Make sure to use numbers strictly within $[-1, 1]$ to fit within the clipping volume in normalised device coordinates.) You will need eight vertices for this.

```
float points[] = {
  // A cube has 8 vertices...
  -0.5, -0.5, -0.5, // vertex 0
  -0.5, -0.5,  0.5, // vertex 1
  -0.5,  0.5, -0.5, // vertex 2
  // etc...
```

The cube will need $2 \times 6 =$ 12 triangular faces. This object has more faces than vertices (as opposed to the single triangle from before, which had three vertices and only one face), because every adjacent face shares vertices with its neighbours. To define all faces, we therefore use an *index array* to specify which three vertices will compose each triangle, as follows:

```
unsigned short faces[]= {
    // ... and 12 triangular faces, defined by these vertex indices:
    0, 3, 2, // triangle 0
    0, 1, 3,
    // etc
  };
```

When specifying the order of the vertex indices in the index array, it is important the the vertices that make up one triangle are listed in *counter-clockwise* order, as viewed from the outside of the object. In this way you specify the "front" and "back" of a triangle, and since OpenGL may skip rendering triangles that are seen from the back (for efficiency reasons) you might end up with a black window or unexpected results if you do it in the wrong order. Please refer to Figure 4 when specifying the order.

In the previous tasks, we didn't have an index array, and it isn't strictly necessary. We could also just create a larger VBO and repeat the vertices so that every three vertices in the vertex buffer results in one triangle. However, that would mean that we would need to transfer redundant vertex information from the CPU to the GPU.

The indices defined in `faces` must also be sent to a GPU buffer before they can be used for drawing. Since this information is not something that is stored for each vertex, we cannot use a *vertex* buffer object for this. Instead, we have to use another construct called an *element* buffer object (EBO).[3] Using these indexes to specify what to draw, instead of using a "naked" vertex buffer object, is called *indexed drawing*.

Tasks

1. Define your geometry (using the points and face indices above).
2. Create a VAO, and set up the vertex attribute related to the position of the vertices, just like you have done before.
3. Create an element buffer object (EBO) with the index array `faces`. This is done similarly to how you create the VBO. First run `glGenBuffers` as before but with a new handle to an EBO. Bind it with `glBindBuffer`. The *target* will be different this time: this buffer object doesn't store vertex attributes, but *vertex array indices*. Then, buffer data to your EBO. This is done with `glBufferData` as before. (Note that the data that you are buffering now is the list of indices that make up the triangle faces; that is, the array `faces`. Use the same kind of *target* as for `glBindBuffer`.)
4. The draw call now needs to be different from the previous tasks, since we now do indexed drawing. That is, we no longer draw using the vertex attribute array directly, but using the element buffer object. So instead of `glDrawArrays`, we need to call `glDrawElements`.

   The documentation for `glDrawElements` might be a little confusing. Please note that *count* refers to the total number of *indices* to be rendered, so 3 for each face. The pointer *indices* can be set to zero, to start rendering from the first index in the EBO. (And for *type*, note that `faces` in the example above is defined as an array of `unsigned short`.)
5. Modify your fragment shader to display a color related to the z-value of the position forwarded from the vertex shader. Note that you may need to flip the sign to get a positive value. (Negative values will appear as black, so you might not see anything!).

So far, since we are looking at a cube straight ahead, it looks only like a flat rectangle (as in Figure 5). You may also note that the object is scaled/stretched as you resize the window, so it is not actually a cube with square sides.

Why is that? It is because we are specifying all coordinates directly as *normalised device coordinates*, and there is no notion of a world/object/view space. In the next task, we will take care of that!

Mandatory 1.5 Model, view & projection transformations

[3] https://www.opengl.org/wiki/Vertex_Specification#Index_buffers

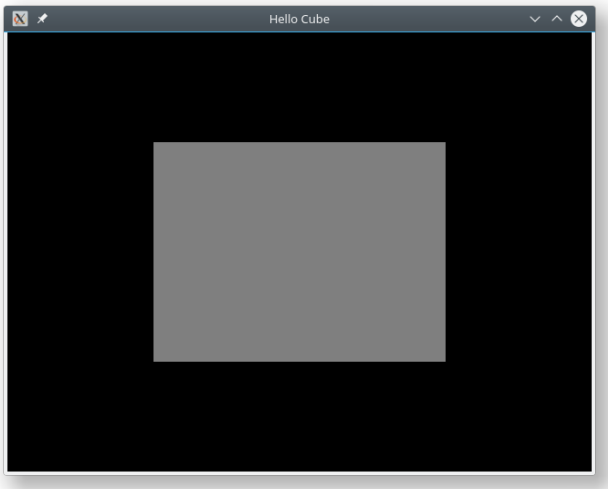


Figure 5: A flat cube (?) from Lab 1.4.

Until now we have not bothered with the position of the geometry in the *scene*, but defined coordinates directly in the clipping volume defined in normalised device coordinates. We have also done nothing to specify the type of camera we are using, or determined the pose of the camera. In this section we will deal with the these issues by defining the pose of the model in the world, the pose of the camera, and the camera's projective properties; and importantly, how to transform coordinates from a scene perspective to the rendering volume that OpenGL ultimately uses to draw the scene.

Any rigid-body transformation (i e, one that changes the position and orientation of the geometry) can be described as the combined effects of rotating and translating the geometry:

$$\vec{v}_b = \vec{t} + \mathbf{R}\vec{v}_a, \tag{1}$$

where $\vec{v}_a$ and $\vec{v}_b$ are, respectively, the initial and final positions of a vertex, $\vec{t}$ is a translation vector and $\mathbf{R}$ is a 3×3 rotation matrix.

Recall from Lecture 2 that the operations of translating and rotating the geometry can be combined into a single matrix-vector multiplication by writing $\vec{v}$ in homogeneous coordinates and combining the rotation and translation into a 4×4 matrix

$$\boldsymbol{T} = \begin{pmatrix} R_{1,1} & R_{1,2} & R_{1,3} & t_1 \\ R_{2,1} & R_{2,2} & R_{2,3} & t_2 \\ R_{3,1} & R_{3,2} & R_{3,3} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{2}$$

Please note that the skeleton vertex shader now has `in vec4 vp` instead of `in vec3 vp`. This is needed because we need a 4D vector when multiplying with a homogeneous 4×4 matrix. You *might* want to change the geometry definition on the CPU side accordingly, to have 4D vectors $(x, y, z, 1)$ for all vertices (with $w = 1$ for positions, as explained in Lecture 2). However, passing a 3D vector into a `vec4` in GLSL works too, in this case, since the fourth component is automatically filled in with 1. Later on, when we also include normal vectors (that is, directions), it will become more important to explicitly handle 4D vectors and set the w component.

In graphics, we typically deal with a sequence of three transformations: first *model*, then *view*, and then *projection*. Given an object (i e, a model) that is defined in some local object frame, the model transformation moves all points of the object to where they are relative to the world coordinate frame. After applying the view transformation, we now have the object position as seen from the camera. Finally, a projection transformation ensures that things that are far from the camera look smaller. What we want to render is the object geometry after applying all of these transformations.

The view matrix transforms everything so that all coordinates are relative to the camera position and orientation. In other words, after applying the view matrix, we are now in a coordinate frame where the camera position is at $(0, 0, 0)$.

The projection matrix is another 4×4 matrix that scales far away objects down to look smaller, and also makes points fit inside *normalised device coordinates* (so that all coordinates are between -1 and $+1$).

In this task you will perform the necessary transformations by hand in order to fully see what goes on "under the hood." In later tasks, you will use library functions instead, which make these operations less tedious.

Tasks

1. Similar to what you did in Lab 1.3, set up a variables that can hold two rotation angles, and change them in the `key_callback` function, so that the object can be rotated with the arrow keys. (E g, rotate around the x axis with left/right, and around the y axis with up/down.)

2. Define two 4×4 matrices `GLfloat rotate_x[4][4]` and `GLfloat rotate_y[4][4]`. (Recall from the book, RTR4 Chapter 4.1.2, how to construct a rotation matrix.) Use your rotation variable from above to set the angle of the rotation matrix.

3. Define a 4×4 matrix `GLfloat modelMatrix[4][4]`

4. Fill in the function `MUL_4x4` that multiplies two 4×4 matrices.

5. Set `modelMatrix` to the product of two transformation matrices: one applying a rotation around the x axis, the other applying a rotation around the y axis.

6. Define a `GLfloat viewMatrix[4][4]` that translates the camera along the z axis by 2 units. (Recall from the book or the lecture how a translation matrix looks.) In other words, the camera should be at world position $(0, 0, 2)$. Now, when transforming your points into the view space, you will need to apply the *inverse* of this matrix. You don't have to write a general-purpose matrix inverse function, but for this simple translation-only matrix, you simply need to *negate the translation*.

7. Define a `GLfloat projectionMatrix[4][4]` with parameters `near = 1.0` and `far = 100.0` (so that the distance to the near clip plane is one unit). See RTR4 Chapter 4.7.2 and the slides from Lecture 2 for information on how to construct such a matrix. (Note that you might get strange effects if you use integer division!)

8. Pass your transformation matrices to uniforms in the shader program. You can set up `uniform mat4 modelMatrix` etc, or a combined matrix `uniform mat4 modelViewMatrix` or `uniform mat4 modelViewProjectionMatrix`.

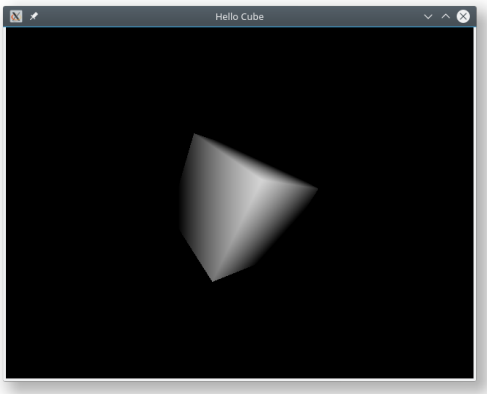


Figure 6: Cube from Lab 1.5, now with model, view and projection transformations applied. Compared to Figure 5, this has a correct perspective and aspect ratio.

Either pass each matrix separately and multiply them together in the vertex shader, or multiply them on the CPU side and send the combined matrix to the vertex shader. (Which is more efficient, do you think?)

Note that in `glUniformMatrix4fv`, there is a boolean flag *transpose*. If you, e.g., define a translation matrix as

```
GLfloat viewMatrix[4][4]=
  {
    {1.0f,  0.0f,  0.0f,  0.0f},
    {0.0f,  1.0f,  0.0f,  0.0f},
    {0.0f,  0.0f,  1.0f, -2.0f},
    {0.0f,  0.0f,  0.0f,  1.0f}
  };
```

and if you left-multiply the matrix with the position vector in the vertex shader ("new position = matrix × old position"), then your matrix will need to be transposed when sending it to OpenGL.

To get a pointer to the array that defines your matrix, as the last argument to `glUniformMatrix4fv`, use `&modelViewProjectionMatrix[0][0]`.

9. In your vertex shader, apply the transformation to the vertex (note that OpenGL handles both `mat4-mat4` and `mat4-vec4` multiplications natively using the "`*`" operator).

   The result should look something like Figure 6.

Mandatory

1.6 Outsourcing the linear algebra work

As you perform more and more advanced math operations you will find it increasingly tedious to implement them by hand. To make the code more readable and easier to use, it is therefore recommended to use GLM (OpenGL Mathematics). This is an open-source headers-only C++ implementation of the GLSL primitives (`vec2`, `vec3`, etc.).

For Windows users, GLM is provided with the skeleton code. On Linux, you can install the `libglm-dev` package.

You can find documentation for GLM at https://glm.g-truc.net/0.9.9/api/index.html. Specifically, the code samples in Section 8 of the manual are useful for this task.

Tasks

1. Replace your own matrix functions from Lab 1.5 with those provided by GLM and your array-based matrices with `glm::mat4` objects.

   Create a projection matrix `glm::mat4 Projection` with a call to the function `glm::perspective`, which specifies the viewing angle, the width and height of the window, and the near and far clipping planes. Note that the aspect ratio is specified using width/height in the `glm::perspective` constructor, instead of height/width.

   Create `glm::vec3 rotate` to hold the $x/y/z$ rotations for the model matrix.

   Create `glm::vec3 translate` to hold the $x/y/z$ translation for the view matrix.

   Create a view matrix with `glm::mat4 View(1.0f);` and apply your translation with `glm::mat4 View = glm::translate( View, translate)`.

   Create a model matrix in the same way and apply the x/y rotations by `Model = glm::rotate( Model, rotate.x, glm::vec3(1.0f, 0.0f, 0.0f))`, etc.

2. Upload your resulting modelViewProjection matrix to the shader as a `uniform`. Since your data is now stored as a `glm::mat` instead of a plain C-style array, you cannot just pass the address of your matrix as you might have done earlier. Instead, use `glm::value_ptr` to get a pointer to the matrix data. When using GLM, you do not need to transpose the matrix when passing it to OpenGL. (In other words, the *transpose* flag of `glUniformMatrix4fv` should be `GL_FALSE`, and the pointer in the last argument should be the what you get from `glm::value_ptr`.)

+1 point 1.7 Moving the camera

So far, the camera has been in a fixed position at $(0, 0, 2)$ in world coordinates. Let's make a more interesting view matrix, so we can move it around. Typically, this is done by specifying a view matrix in world coordinates (the position and orientation of the camera in the world) and then applying the inverse of that matrix in the MVP transformation (to transform coordinates to where they are in the camera's point of view).

Similar to what you have done for rotating the cube with the keyboard, set up a transformation for the camera. You can have variables for the camera rotation angles and translation offsets that are modified with some other keys in the `key_callback` function, or if you want, rotate/translate with the mouse (check the GLFW documentation for mouse input in that case).

Implement functionality so that you can both translate and rotate the camera, and rotate the cube independently.

Take care that the order of applying the transformations to your GLM matrices is correct. Note that the order of calling `glm::rotate` and `glm::translate` is the same as the operations would be applied with matrix multiplication from the left. That is, the left-most matrix (the first `glm` transformation call) happens last, and the right-most (the last `glm` transformation call) happens

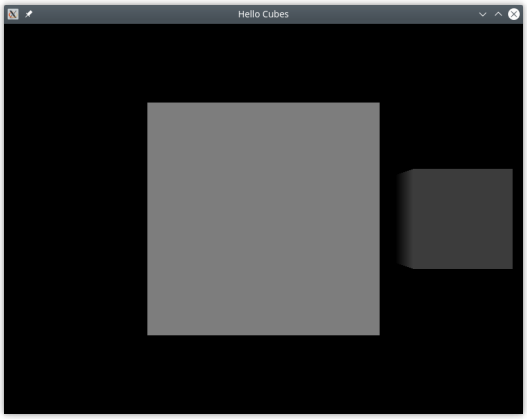


Figure 7: Two cubes from Lab 1.8.

first. How should you set up the calls to offset the camera from the origin and rotate it around its own axis, versus rotating it around the origin so that it orbits around the cube?

Copy your code from Lab 1.6 to `lab1-7.cpp` and update it.

+1 point

1.8 Multiple cubes

Let's make a second cube, with its own Model matrix, that can also be moved around independently of the original cube and the camera.

- Set up a new array (e.g., `points2`) with vertex coordinates. For example, you can set coordinates for a smaller cube that is offset a bit in the x direction, to make it look like Figure 7.
- Set up a new VAO and a new VBO, and set them up for the new geometry in the same way as you have done before.
- You can reuse the same EBO, if you still have a cube with vertices connected in the same order. Connect the EBO to your new VAO.
- In the main `while` loop, you should now do *two* draw calls. First, compute and upload the MVP matrix for the first object, bind one VAO, and call `glDrawElements`. Then, compute and upload the MVP matrix for the second object, bind the other VAO, and call `glDrawElements` again. These two draw calls both update the framebuffer with your new image. Finally, the call to `glfwSwapBuffers` pushes the image after your draw calls are complete to the window on the screen.

The instructions above is just one way of implementing these two objects. You could also have put all the data in one VBO (with the points for the second cube after the points for the first one), etc.

+1 point

1.9 CUDA

GPUs are essentially massively parallel computers that are good for many other things than producing graphics. In this task you will get to work a bit with *computations* on the GPU – without doing any graphics – using the *CUDA* platform.

CUDA makes it possible to use GPUs for general purpose computing. It provides extensions for C/C++, so you can mix functions that are run on the GPU with regular "CPU-style" C code. The CPU system is referred to as the *host* and the GPU as the *device*.

Please note that CUDA requires an nVidia GPU. That is available on the computers in T006, in case your own computer has a different brand of GPU.

Setup

- On Linux, `sudo apt install nvidia-cuda-toolkit`
- On Windows, assuming the setup in T006, you should run the provided file `vcvars32+nvcc.bat` in a command window, which sets up the paths for the MSVC C++ compiler `cl` as well as the CUDA compiler `nvcc`.

  On your own Windows computer, you may have to install the NVIDIA GPU Computing Toolkit and set up the paths in a similar way as it is done `vcvars32+nvcc.bat`. (This file is a copy of the `vcvars32.bat` that should come with your Visual Studio version, and additionally adds `extras/CUPTI/lib64` to the path.)

  If you don't have a recent VS version (for example in T006, where we only have VS 2015), you should add `-allow-unsupported-compiler` when compiling: `nvcc -allow-unsupported-compiler add.cu`.

Tasks

In this task, you should implement the steps of the CUDA tutorial at https://cuda-tutorial.readthedocs.io/en/latest/tutorials/tutorial01/

- Look at the C file `lab1-9.c`. It adds two (*long*) arrays and stores result in third array.
- Compile the code and run it, and verify that the addition is performed correctly (check the printouts of the first and last element in the arrays).

Question 1 What output do you get?

- Now, let's move some of the work from the CPU to the GPU. Go to the section *Exercise: Converting vector addition to CUDA* in the tutorial.

  Copy `lab1-9.c` to `lab1-9.cu`. Convert the C `vector_add` function to a GPU *kernel* as per the instructions in the tuturial: just by adding a `__global__` prefix. (It still uses same C syntax.)
- In the `main` function in `lab1-9.cu`, you should now push the kernel and execute it on the GPU, instead of running the C function, as per the instructions in the tutorial. (This is done by adding `<<<1,1>>>` to the function call from main.)
- Compile the program with the CUDA compiler:
  `nvcc lab1-9.cu -o build/lab1-9`
  or `nvcc lab1-9.cu -o -allow-unsupported-compiler build/lab1-9`.
- Run the program. Check the output, and note that the computation is no longer correct. This is because the GPU cannot access the array we have in CPU RAM.

Question 2 What output do you get now?

- In order to get the data to the GPU, you must do something akin to the "buffer data" call for the VBO in OpenGL, although with CUDA the calls are a bit different from what they are in OpenGL. As noted in the tutorial, what is currently missing is to allocate *device memory* (a "buffer object"), transfer data from the host to the device ("buffer data"), and transfer data back from host to device after we have executed the GPU kernel function.
- The CUDA way of transferring data is to run `cudaMalloc` and then `cudaMemcpy` (and `cudaFree`). Note the similarity to standard C memory management functions.
- You already have three *host* arrays `a, b, out`. Add the corresponding *device* arrays. In the C code, they will look the same: still just a float pointer `float *a_device;`. Note the syntax of the `cudaMalloc` and `cudaMemcpy` functions. For `cudaMalloc`, the first argument is a *pointer to your device pointer* and the second argument is the number of bytes to allocate. For `cudaMemcpy`, the arguments are: the destination pointer, the source pointer, number of bytes to copy, and a direction flag that should be either `cudaMemcpyDeviceToHost` or `cudaMemcpyHostToDevice`.

Question 3 Which array must be transferred before and after kernel execution? Why? Which pointers should you pass to `vector_add`?

Question 4 Which result do you get now? Is it the same as the original CPU version?

+1 point

1.10 Additional questions

This task is to answer some questions about the previous exercises in this lab.

Question 5

- What are the limits for x and y in Lab 1.1, and what happens if you exceed them? What are the limits for z and what happens if you exceed them? Why? (It has to do with normalised device coordinates.)

Question 6

- If you didn't use an element buffer array in Lab 1.4, how would you have set up your VBO instead?
- Print out the model, view, projection matrices from Lab 1.6:
  `std::cout << glm::to_string(Projection)<< std::endl;`

  Please note that the printout from `glm::to_string` will appear transposed, compared to the matrices shown in the lecture and the book.

Question 7 What does the model matrix look like? Describe the roles of the different matrix elements. For each non-zero element, can you explain what it does in terms of scaling, rotating translating, etc?

Question 8 What does the view matrix look like? Describe the roles of the different matrix elements. For each non-zero element, can you explain what it does in terms of scaling, rotating translating, etc?

Question 9 What does the projection matrix look like? Describe the roles of the different matrix elements. For each non-zero element, can you explain what it does in terms of scaling, rotating translating, etc?

Question 10

- So far the `glDepthFunc` has been set to `GL_LESS`. What other options can you set for this parameter? Is the original choice appropriate? Why? What can happen otherwise?

Outro

This marks the end of the first set of lab assignments. By now, you have learnt the mechanics of how to pass data from the CPU to the GPU with OpenGL and how to work with 3D transformation matrices. In the next lab, you will prettify things by experimenting with more advanced materials and lighting.